

COMPLETE CSS HANDBOOK

TABLE OF CONTENTS

1. ANIMATIONS

2. BACKGROUND

3. BASICS

4. BORDERS

5. FLEX-BOXES

6. FLOAT

7. FONTS

8. GLOWING BUTTON V2

9. GLOWING BUTTONS EFFECT

10. GLOWING NAVBAR V1

11. GRID

12. GRIDS

13. MARGINS

14. POSITION

15. PSEUDO CLASSES

1. ANIMATIONS

@keyframes Rule

Basic Syntax:

```
@keyframes animationName {
  0% {
    /* start state */
  }
  50% {
    /* middle state */
  }
  100% {
    /* end state */
  }
}
```

Alternative Syntax:

```
@keyframes animationName {
  from {
    /* start state */
  }
  to {
    /* end state */
  }
}
```

Animation Properties

Core Animation Properties

animation-name

- Purpose: Specifies the name of the keyframe animation
- Values: animationName | none
- Example: animation-name: slideIn;

animation-duration

- Purpose: Duration of the animation
- Values: time (seconds or milliseconds)
- Example: animation-duration: 2s; or animation-duration: 500ms;

animation-timing-function

- Purpose: Speed curve of animation
- Values:
 - `linear` - Same speed from start to end
 - `ease` - Slow start, then fast, then slow end (default)
 - `ease-in` - Slow start
 - `ease-out` - Slow end
 - `ease-in-out` - Slow start and end
 - `cubic-bezier(n,n,n,n)` - Custom timing function
- Example: `animation-timing-function: ease-in-out;`

animation-delay

- Purpose: Delay before animation starts
- Values: `time` (seconds or milliseconds)
- Example: `animation-delay: 1s;`

animation-iteration-count

- Purpose: Number of times animation repeats
- Values: `number` | `infinite`
- Example: `animation-iteration-count: 3;` or `animation-iteration-count: infinite;`

animation-direction

- Purpose: Direction of animation
- Values:
 - `normal` - Forward direction (default)
 - `reverse` - Backward direction
 - `alternate` - Forward then backward
 - `alternate-reverse` - Backward then forward
- Example: `animation-direction: alternate;`

animation-fill-mode

- Purpose: What happens before/after animation
- Values:
 - `none` - No styles applied before/after (default)
 - `forwards` - Keeps final keyframe styles
 - `backwards` - Applies first keyframe styles before animation
 - `both` - Applies both forwards and backwards
- Example: `animation-fill-mode: forwards;`

animation-play-state

- Purpose: Whether animation is running or paused
- Values: `running` | `paused`
- Example: `animation-play-state: paused;`

Animation Shorthand Property

Syntax:

```
animation: name duration timing-function delay iteration-count direction fill-mode play-state;
```

Example:

```
animation: slideIn 2s ease-in-out 0.5s infinite alternate forwards running;
```

Transform Properties (Used with Animations)

2D Transforms

transform

- Purpose: Applies 2D or 3D transformations
- Values:
 - `translate(x, y)` - Move element
 - `translateX(value)` - Move horizontally
 - `translateY(value)` - Move vertically
 - `rotate(angle)` - Rotate element
 - `scale(x, y)` - Scale element
 - `scaleX(value)` - Scale horizontally
 - `scaleY(value)` - Scale vertically
 - `skew(x-angle, y-angle)` - Skew element
 - `skewX(angle)` - Skew horizontally
 - `skewY(angle)` - Skew vertically

Examples:

```
transform: translateX(100px);
transform: rotate(45deg);
transform: scale(1.5, 2);
transform: skew(30deg, 20deg);
```

3D Transforms

3D Transform Functions:

- `translateZ(value)` - Move along Z-axis
- `rotateX(angle)` - Rotate around X-axis
- `rotateY(angle)` - Rotate around Y-axis
- `rotateZ(angle)` - Rotate around Z-axis
- `scaleZ(value)` - Scale along Z-axis

Examples:

```
transform: translateZ(100px);
transform: rotateX(45deg);
transform: rotateY(90deg);
```

Transform Origin

transform-origin

- Purpose: Origin point for transformations
- Values: `x-axis y-axis z-axis`
- Keywords: `left, center, right, top, bottom`
- Example: `transform-origin: top left;`

3D Transform Properties

transform-style

- Purpose: How nested elements are rendered in 3D space
- Values: `flat | preserve-3d`
- Example: `transform-style: preserve-3d;`

perspective

- Purpose: Distance between z=0 and user
- Values: `none | length`
- Example: `perspective: 1000px;`

perspective-origin

- Purpose: Position of perspective origin
- Values: `x-axis y-axis`
- Example: `perspective-origin: 50% 50%;`

backface-visibility

- Purpose: Whether back face is visible
- Values: `visible | hidden`
- Example: `backface-visibility: hidden;`

Transition Properties

transition-property

- Purpose: CSS property to transition
- Values: `property-name | all | none`
- Example: `transition-property: background-color;`

transition-duration

- Purpose: Duration of transition
- Values: `time`
- Example: `transition-duration: 0.3s;`

transition-timing-function

- Purpose: Speed curve of transition
- Values: Same as animation-timing-function
- Example: `transition-timing-function: ease-in-out;`

transition-delay

- Purpose: Delay before transition starts
- Values: time
- Example: `transition-delay: 0.1s;`

transition (Shorthand)

- Syntax: `property duration timing-function delay`
- Example: `transition: all 0.3s ease-in-out 0.1s;`

Animation Examples

Fade In Animation:

```
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

.fade-in {
  animation: fadeIn 1s ease-in;
}
```

Slide In Animation:

```
@keyframes slideIn {
  from { transform: translateX(-100%); }
  to { transform: translateX(0); }
}

.slide-in {
  animation: slideIn 0.5s ease-out;
}
```

Bounce Animation:

```
@keyframes bounce {
  0%, 20%, 50%, 80%, 100% { transform: translateY(0); }
  40% { transform: translateY(-30px); }
  60% { transform: translateY(-15px); }
}

.bounce {
  animation: bounce 1s infinite;
}
```

2. BACKGROUND

Background Color

background-color

- Purpose: Sets background color
- Values: color | transparent
- Examples:
 - `background-color: red;`
 - `background-color: #FF0000;`
 - `background-color: rgb(255, 0, 0);`
 - `background-color: rgba(255, 0, 0, 0.5);`

Background Image

background-image

- Purpose: Sets background image

- Values:
 - `url(image-path)` - Image file
 - `linear-gradient()` - Linear gradient
 - `radial-gradient()` - Radial gradient
 - `none` - No image
- Examples:
 - `background-image: url('image.jpg');`
 - `background-image: linear-gradient(to right, red, blue);`

Background Repeat

background-repeat

- Purpose: How background image repeats
- Values:
 - `repeat` - Repeat both horizontally and vertically (default)
 - `repeat-x` - Repeat horizontally only
 - `repeat-y` - Repeat vertically only
 - `no-repeat` - Don't repeat
 - `space` - Repeat with spaces between images
 - `round` - Repeat and stretch to fit
- Example: `background-repeat: no-repeat;`

Background Position

background-position

- Purpose: Position of background image
- Values:
 - Keywords: `left`, `center`, `right`, `top`, `bottom`
 - Percentages: `0%` to `100%`
 - Length units: `px`, `em`, etc.
 - Combinations: `x-position y-position`
- Examples:
 - `background-position: center;`
 - `background-position: top right;`
 - `background-position: 50% 25%;`
 - `background-position: 100px 200px;`

Background Size

background-size

- Purpose: Size of background image
- Values:
 - `auto` - Original size (default)
 - `cover` - Scale to cover entire container
 - `contain` - Scale to fit within container
 - `length` - Specific size (e.g., `100px`)
 - `percentage` - Percentage of container
 - `width height` - Specific width and height
- Examples:
 - `background-size: cover;`
 - `background-size: 100% 50%;`
 - `background-size: 300px 200px;`

Background Attachment

background-attachment

- Purpose: Whether background scrolls or fixed
- Values:
 - `scroll` - Scrolls with content (default)
 - `fixed` - Fixed to viewport
 - `local` - Scrolls with element's content
- Example: `background-attachment: fixed;`

Background Origin

background-origin

- Purpose: Origin of background position
- Values:
 - `padding-box` - Relative to padding box (default)
 - `border-box` - Relative to border box

- `content-box` - Relative to content box
- Example: `background-origin: border-box;`

Background Clip

background-clip

- Purpose: Clipping area of background
- Values:
 - `border-box` - Clip to border box (default)
 - `padding-box` - Clip to padding box
 - `content-box` - Clip to content box
 - `text` - Clip to text (creates text effects)
- Example: `background-clip: padding-box;`

Background Shorthand

Syntax:

```
background: color image repeat attachment position / size;
```

Examples:

```
background: red url('image.jpg') no-repeat fixed center / cover;
background: linear-gradient(45deg, red, blue) no-repeat;
background: #fff url('pattern.png') repeat-x top;
```

Gradient Functions

Linear Gradient

Basic Syntax:

```
background: linear-gradient(direction, color1, color2, ...);
```

Direction Options:

- `to top`, `to bottom`, `to left`, `to right`
- `to top left`, `to top right`, `to bottom left`, `to bottom right`
- Angle: `45deg`, `90deg`, `180deg`, etc.

Examples:

```
background: linear-gradient(to right, red, blue);
background: linear-gradient(45deg, red, yellow, blue);
background: linear-gradient(to bottom, #ff0000, #00ff00, #0000ff);
```

Color Stops:

```
background: linear-gradient(to right,
  red 0%,
  yellow 50%,
  blue 100%
);
```

Radial Gradient

Basic Syntax:

```
background: radial-gradient(shape size at position, color1, color2, ...);
```

Shape Options:

- `circle` - Circular gradient
- `ellipse` - Elliptical gradient (default)

Size Options:

- `closest-side` - Gradient ends at closest side
- `farthest-side` - Gradient ends at farthest side
- `closest-corner` - Gradient ends at closest corner
- `farthest-corner` - Gradient ends at farthest corner (default)

Position Options:

- Keywords: `center`, `top`, `bottom`, `left`, `right`
- Percentages: `50%` `50%`
- Length units: `100px` `200px`

Examples:

```
background: radial-gradient(circle, red, blue);
background: radial-gradient(ellipse at top left, red, blue);
background: radial-gradient(circle at 25% 25%, red, yellow, blue);
```

Multiple Backgrounds

Syntax:

```
background: image1, image2, image3, color;
```

Example:

```
background:
  url('foreground.png') no-repeat center,
  url('background.jpg') no-repeat center / cover,
  linear-gradient(45deg, red, blue);
```

3. BASICS

CSS Selectors

Universal Selector

Syntax: `*`

- Purpose: Selects all elements
- Example: `* { margin: 0; padding: 0; }`

Element Selector

Syntax: `element`

- Purpose: Selects all elements of specified type
- Examples:
 - `p { color: blue; }`
 - `div { background: red; }`
 - `h1 { font-size: 24px; }`

Class Selector

Syntax: `.class`

- Purpose: Selects elements with specified class
- Examples:
 - `.container { width: 100%; }`
 - `.button { padding: 10px; }`
 - `.red-text { color: red; }`

ID Selector

Syntax: `#id`

- Purpose: Selects element with specified ID
- Examples:
 - `#header { background: blue; }`

- `#main-content { margin: 20px; }`
- `#footer { text-align: center; }`

Attribute Selector

Syntax: `[attribute]` or `[attribute="value"]`

- Purpose: Selects elements with specified attribute
- Examples:
 - `[href] { color: blue; }`
 - `[type="text"] { border: 1px solid black; }`
 - `[class*="btn"] { padding: 5px; }`

Attribute Selector Variations:

- `[attr="value"]` - Exact match
- `[attr~="value"]` - Word match
- `[attr|="value"]` - Prefix match
- `[attr^="value"]` - Starts with
- `[attr$="value"]` - Ends with
- `[attr*="value"]` - Contains

Combinators

Descendant Selector: `ancestor descendant`

- Purpose: Selects elements that are descendants
- Example: `div p { color: red; }`

Child Selector: `parent > child`

- Purpose: Selects direct children
- Example: `ul > li { list-style: none; }`

Adjacent Sibling: `element + sibling`

- Purpose: Selects immediately following sibling
- Example: `h1 + p { margin-top: 0; }`

General Sibling: `element ~ sibling`

- Purpose: Selects all following siblings
- Example: `h1 ~ p { color: gray; }`

CSS Units

Absolute Units

px (Pixels)

- Purpose: Fixed unit based on screen pixels
- Example: `width: 300px;`

pt (Points)

- Purpose: 1/72 of an inch
- Example: `font-size: 12pt;`

pc (Picas)

- Purpose: 12 points
- Example: `width: 2pc;`

in (Inches)

- Purpose: Physical inches
- Example: `width: 2in;`

cm (Centimeters)

- Purpose: Physical centimeters
- Example: `width: 5cm;`

mm (Millimeters)

- Purpose: Physical millimeters
- Example: `width: 50mm;`

Relative Units

em

- Purpose: Relative to parent element's font size
- Example: `font-size: 1.2em;`

rem

- Purpose: Relative to root element's font size
- Example: `font-size: 1.5rem;`

% (Percentage)

- Purpose: Relative to parent element
- Example: `width: 50%;`

vh (Viewport Height)

- Purpose: 1% of viewport height
- Example: `height: 100vh;`

vw (Viewport Width)

- Purpose: 1% of viewport width
- Example: `width: 50vw;`

vmin (Viewport Minimum)

- Purpose: 1% of viewport's smaller dimension
- Example: `font-size: 4vmin;`

vmax (Viewport Maximum)

- Purpose: 1% of viewport's larger dimension
- Example: `font-size: 4vmax;`

Color Properties

Color Property

color

- Purpose: Sets text color
- Values: Any valid color value
- Example: `color: red;`

Opacity Property

opacity

- Purpose: Sets transparency level
- Values: 0 (transparent) to 1 (opaque)
- Example: `opacity: 0.5;`

Color Value Types

Named Colors

- Examples: `red`, `blue`, `green`, `black`, `white`
- Usage: `color: red;`

Hexadecimal Colors

- 6-digit: `#RRGGBB` (e.g., `#FF0000`)
- 3-digit: `#RGB` (e.g., `#F00`)
- 8-digit with alpha: `#RRGGBBAA`
- Examples: `color: #FF0000;`, `color: #F00;`

RGB Colors

- Syntax: `rgb(red, green, blue)`
- Values: 0-255 for each channel
- Example: `color: rgb(255, 0, 0);`

RGBA Colors

- Syntax: `rgba(red, green, blue, alpha)`
- Alpha: 0-1 for transparency
- Example: `color: rgba(255, 0, 0, 0.5);`

HSL Colors

- Syntax: `hsl(hue, saturation, lightness)`
- Hue: 0-360 degrees
- Saturation: 0-100%
- Lightness: 0-100%
- Example: `color: hsl(0, 100%, 50%);`

HSLA Colors

- Syntax: `hsla(hue, saturation, lightness, alpha)`
- Example: `color: hsla(0, 100%, 50%, 0.5);`

Display Properties

Display Property

display

- Purpose: Defines how element is displayed
- Values:
 - `block` - Block-level element
 - `inline` - Inline element
 - `inline-block` - Inline block element
 - `flex` - Flexible box container
 - `grid` - Grid container
 - `table` - Table element
 - `none` - Element not displayed
 - `contents` - Element's box disappears

Examples:

```
.block { display: block; }  
.inline { display: inline; }  
.flex-container { display: flex; }  
.hidden { display: none; }
```

Visibility Properties

Visibility Property

visibility

- Purpose: Controls element visibility
- Values:
 - `visible` - Element is visible (default)
 - `hidden` - Element is hidden but space is reserved
 - `collapse` - For table elements
- Example: `visibility: hidden;`

Difference between `display: none` and `visibility: hidden`:

- `display: none` - Element is completely removed
- `visibility: hidden` - Element is hidden but space is preserved

Overflow Properties

Overflow Property

overflow

- Purpose: Controls what happens when content overflows
- Values:
 - `visible` - Content is not clipped (default)
 - `hidden` - Content is clipped
 - `scroll` - Scrollbars are added
 - `auto` - Scrollbars appear if needed
- Example: `overflow: hidden;`

overflow-x

- Purpose: Horizontal overflow behavior
- Values: Same as `overflow`
- Example: `overflow-x: scroll;`

overflow-y

- Purpose: Vertical overflow behavior
- Values: Same as `overflow`
- Example: `overflow-y: auto;`

Box Model Properties

Width and Height

width

- Purpose: Sets element width
- Values: length, percentage, auto
- Example: width: 300px;

height

- Purpose: Sets element height
- Values: length, percentage, auto
- Example: height: 200px;

min-width / max-width

- Purpose: Sets minimum/maximum width
- Example: min-width: 200px; max-width: 800px;

min-height / max-height

- Purpose: Sets minimum/maximum height
- Example: min-height: 100px; max-height: 500px;

Box Sizing

box-sizing

- Purpose: Controls how box size is calculated
- Values:
 - content-box - Width/height includes only content (default)
 - border-box - Width/height includes padding and border
- Example: box-sizing: border-box;

4. BORDERS

Border Width

border-width

- Purpose: Sets width of all borders
- Values:
 - Keywords: thin, medium, thick
 - Length: 1px, 2px, 0.5em, etc.
 - Multiple values: top right bottom left
- Examples:
 - border-width: 2px;
 - border-width: 1px 2px 3px 4px;

Individual Border Widths:

- border-top-width
- border-right-width
- border-bottom-width
- border-left-width

Border Style

border-style

- Purpose: Sets style of all borders
- Values:
 - none - No border
 - solid - Solid line
 - dashed - Dashed line
 - dotted - Dotted line
 - double - Double line
 - groove - 3D grooved border
 - ridge - 3D ridged border
 - inset - 3D inset border
 - outset - 3D outset border
 - hidden - Same as none
- Examples:
 - border-style: solid;
 - border-style: dashed dotted;

Individual Border Styles:

- `border-top-style`
- `border-right-style`
- `border-bottom-style`
- `border-left-style`

Border Color

border-color

- Purpose: Sets color of all borders
- Values: Any valid color value
- Examples:
 - `border-color: red;`
 - `border-color: #FF0000;`
 - `border-color: rgb(255, 0, 0);`

Individual Border Colors:

- `border-top-color`
- `border-right-color`
- `border-bottom-color`
- `border-left-color`

Border Shorthand

border

- Purpose: Sets width, style, and color of all borders
- Syntax: `border: width style color;`
- Examples:
 - `border: 2px solid red;`
 - `border: 1px dashed #333;`
 - `border: 3px double blue;`

Individual Border Sides:

- `border-top: width style color;`
- `border-right: width style color;`
- `border-bottom: width style color;`
- `border-left: width style color;`

Border Radius

border-radius

- Purpose: Creates rounded corners
- Values:
 - Single value: `border-radius: 10px;`
 - Two values: `border-radius: 10px 20px;` (horizontal vertical)
 - Four values: `border-radius: 10px 20px 30px 40px;` (top-left, top-right, bottom-right, bottom-left)
- Examples:
 - `border-radius: 50%;` (creates circle)
 - `border-radius: 10px;` (rounded corners)
 - `border-radius: 10px 20px;` (elliptical corners)

Individual Corner Radius:

- `border-top-left-radius`
- `border-top-right-radius`
- `border-bottom-right-radius`
- `border-bottom-left-radius`

Examples:

```
.circle { border-radius: 50%; }
.rounded { border-radius: 10px; }
.pill { border-radius: 25px; }
.custom { border-radius: 10px 20px 30px 40px; }
```

Border Images

border-image-source

- Purpose: Sets source image for border
- Values: `url(image)` or `none`
- Example: `border-image-source: url('border.png');`

border-image-slice

- Purpose: How to slice border image
- Values: Numbers or percentages
- Example: `border-image-slice: 30;`

border-image-width

- Purpose: Width of border image
- Values: Length, percentage, or number
- Example: `border-image-width: 10px;`

border-image-outset

- Purpose: Border image outset
- Values: Length or number
- Example: `border-image-outset: 5px;`

border-image-repeat

- Purpose: How border image repeats
- Values:
 - `stretch` - Image is stretched
 - `repeat` - Image is repeated
 - `round` - Image is repeated and scaled
 - `space` - Image is repeated with spacing
- Example: `border-image-repeat: repeat;`

border-image (Shorthand)

- Syntax: `border-image: source slice width outset repeat;`
- Example: `border-image: url('border.png') 30 10px repeat;`

Outline Properties

outline

- Purpose: Creates outline around element (outside border)
- Syntax: `outline: width style color;`
- Examples:
 - `outline: 2px solid red;`
 - `outline: 1px dashed blue;`

Individual Outline Properties:

- `outline-width` - Width of outline
- `outline-style` - Style of outline
- `outline-color` - Color of outline

outline-offset

- Purpose: Distance between outline and border
- Values: Length (can be negative)
- Example: `outline-offset: 5px;`

Note: Outlines do not affect layout and do not have border-radius.

5. FLEX-BOXES

Flex Container Setup

display: flex

- Purpose: Creates a flex container
- Values: `flex` or `inline-flex`
- Example: `display: flex;`

Flex Direction

flex-direction

- Purpose: Defines direction of flex items
- Values:
 - `row` - Left to right (default)
 - `row-reverse` - Right to left
 - `column` - Top to bottom
 - `column-reverse` - Bottom to top
- Example: `flex-direction: column;`

Flex Wrap

flex-wrap

- Purpose: Whether flex items wrap to new lines
- Values:
 - nowrap - All items on one line (default)
 - wrap - Items wrap to new lines
 - wrap-reverse - Items wrap in reverse order
- Example: `flex-wrap: wrap;`

Flex Flow (Shorthand)

flex-flow

- Purpose: Combines flex-direction and flex-wrap
- Syntax: `flex-flow: direction wrap;`
- Examples:
 - `flex-flow: row wrap;`
 - `flex-flow: column nowrap;`

Justify Content (Main Axis)

justify-content

- Purpose: Alignment along main axis
- Values:
 - flex-start - Items at start of container
 - flex-end - Items at end of container
 - center - Items centered
 - space-between - Equal space between items
 - space-around - Equal space around items
 - space-evenly - Equal space between and around items
- Example: `justify-content: center;`

Align Items (Cross Axis)

align-items

- Purpose: Alignment along cross axis
- Values:
 - stretch - Items stretch to fill container (default)
 - flex-start - Items at start of cross axis
 - flex-end - Items at end of cross axis
 - center - Items centered on cross axis
 - baseline - Items aligned along baseline
- Example: `align-items: center;`

Align Content (Multiple Lines)

align-content

- Purpose: Alignment of wrapped lines
- Values:
 - stretch - Lines stretch to fill container (default)
 - flex-start - Lines at start of container
 - flex-end - Lines at end of container
 - center - Lines centered
 - space-between - Equal space between lines
 - space-around - Equal space around lines
 - space-evenly - Equal space between and around lines
- Example: `align-content: space-between;`

Gap Properties

gap

- Purpose: Space between flex items
- Values: Length values
- Example: `gap: 20px;`

row-gap

- Purpose: Space between rows
- Example: `row-gap: 15px;`

column-gap

- Purpose: Space between columns
- Example: `column-gap: 10px;`

Flex Item Properties

Order

order

- Purpose: Controls order of flex items
- Values: Integer (can be negative)
- Default: 0
- Example: `order: 1;`

Flex Grow

flex-grow

- Purpose: How much item should grow
- Values: Number (default: 0)
- Example: `flex-grow: 1;`

Flex Shrink

flex-shrink

- Purpose: How much item should shrink
- Values: Number (default: 1)
- Example: `flex-shrink: 0;`

Flex Basis

flex-basis

- Purpose: Initial size of item before growing/shrinking
- Values: Length, percentage, or `auto`
- Example: `flex-basis: 200px;`

Flex Shorthand

flex

- Purpose: Combines flex-grow, flex-shrink, and flex-basis
- Syntax: `flex: grow shrink basis;`
- Common values:
 - `flex: 1;` = `flex: 1 1 0%;`
 - `flex: auto;` = `flex: 1 1 auto;`
 - `flex: none;` = `flex: 0 0 auto;`

6. FLOAT

Float Property

float

- Purpose: Moves element to left or right, allowing text to wrap around it
- Values:
 - `none` - Element doesn't float (default)
 - `left` - Element floats to the left
 - `right` - Element floats to the right
 - `inherit` - Inherits float value from parent
- Examples:
 - `float: left;`
 - `float: right;`
 - `float: none;`

Clear Property

clear

- Purpose: Prevents element from wrapping around floated elements
- Values:

- none - Allows floating elements on both sides (default)
 - left - No floating elements on left side
 - right - No floating elements on right side
 - both - No floating elements on either side
 - inherit - Inherits clear value from parent
- Examples:
 - clear: both;
 - clear: left;
 - clear: right;

Clearfix Techniques

Traditional Clearfix:

```
.clearfix::after {  
  content: "";  
  display: table;  
  clear: both;  
}
```

Modern Clearfix:

```
.clearfix {  
  overflow: auto;  
}
```

Micro Clearfix:

```
.clearfix::before,  
.clearfix::after {  
  content: "";  
  display: table;  
}  
.clearfix::after {  
  clear: both;  
}
```

Float Layout Examples

Two-Column Layout:

```
.sidebar {  
  width: 30%;  
  float: left;  
}  
.main-content {  
  width: 70%;  
  float: right;  
}  
.container::after {  
  content: "";  
  display: table;  
  clear: both;  
}
```

Three-Column Layout:


```
.left-sidebar {
  width: 20%;
  float: left;
}
.main-content {
  width: 60%;
  float: left;
}
.right-sidebar {
  width: 20%;
  float: right;
}
```

7. FONTS

Font Family

font-family

- Purpose: Specifies font for text
- Values: Font names, generic families
- Generic families: serif, sans-serif, monospace, cursive, fantasy
- Examples:
 - font-family: Arial, sans-serif;
 - font-family: "Times New Roman", serif;
 - font-family: Georgia, "Times New Roman", serif;

Web Safe Fonts:

- Arial, Helvetica, sans-serif
- Georgia, serif
- "Times New Roman", serif
- Courier New, monospace
- Verdana, sans-serif

Font Size

font-size

- Purpose: Sets size of font
- Values:
 - Absolute: xx-small, x-small, small, medium, large, x-large, xx-large
 - Relative: smaller, larger
 - Length: px, em, rem, pt, etc.
 - Percentage: 100%, 120%, etc.
- Examples:
 - font-size: 16px;
 - font-size: 1.2em;
 - font-size: large;

Font Weight

font-weight

- Purpose: Sets thickness of font
- Values:
 - Keywords: normal, bold, bolder, lighter
 - Numbers: 100, 200, 300, 400, 500, 600, 700, 800, 900
 - 400 = normal, 700 = bold
- Examples:
 - font-weight: bold;
 - font-weight: 600;
 - font-weight: lighter;

Font Style

font-style

- Purpose: Sets style of font
- Values:

- `normal` - Normal text (default)
 - `italic` - Italic text
 - `oblique` - Oblique text (slanted)
- Examples:
 - `font-style: italic;`
 - `font-style: normal;`

Font Variant

font-variant

- Purpose: Controls use of small caps
- Values:
 - `normal` - Normal text (default)
 - `small-caps` - Small capital letters
- Example: `font-variant: small-caps;`

Font Stretch

font-stretch

- Purpose: Selects normal, condensed, or expanded face
- Values:
 - `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`
 - `normal`
 - `semi-expanded`, `expanded`, `extra-expanded`, `ultra-expanded`
- Example: `font-stretch: condensed;`

Font Shorthand

font

- Purpose: Sets all font properties
- Syntax: `font: style variant weight size/line-height family;`
- Required: size and family
- Examples:
 - `font: 16px Arial, sans-serif;`
 - `font: bold 18px/1.5 "Times New Roman", serif;`
 - `font: italic small-caps 600 20px Georgia, serif;`

Text Properties

Text Color

color

- Purpose: Sets text color
- Values: Any valid color value
- Examples:
 - `color: red;`
 - `color: #333333;`
 - `color: rgb(50, 50, 50);`

Text Alignment

text-align

- Purpose: Horizontal alignment of text
- Values:
 - `left` - Left-aligned (default)
 - `right` - Right-aligned
 - `center` - Center-aligned
 - `justify` - Justified (full width)
- Examples:
 - `text-align: center;`
 - `text-align: justify;`

Text Decoration

text-decoration

- Purpose: Decorates text
- Values:
 - `none` - No decoration (default)
 - `underline` - Underlined text

- `overline` - Line above text
- `line-through` - Line through text
- `blink` - Blinking text (deprecated)
- Examples:
 - `text-decoration: underline;`
 - `text-decoration: line-through;`

text-decoration-line

- Purpose: Sets decoration line position
- Values: `underline`, `overline`, `line-through`, `none`

text-decoration-color

- Purpose: Sets decoration color
- Values: Any valid color value

text-decoration-style

- Purpose: Sets decoration style
- Values: `solid`, `double`, `dotted`, `dashed`, `wavy`

Text Transform

text-transform

- Purpose: Controls text capitalization
- Values:
 - `none` - No transformation (default)
 - `capitalize` - First letter of each word capitalized
 - `uppercase` - All uppercase
 - `lowercase` - All lowercase
- Examples:
 - `text-transform: uppercase;`
 - `text-transform: capitalize;`

Text Indent

text-indent

- Purpose: Indents first line of text
- Values: Length or percentage
- Examples:
 - `text-indent: 20px;`
 - `text-indent: 2em;`
 - `text-indent: 10%;`

Letter Spacing

letter-spacing

- Purpose: Space between characters
- Values: `normal` or length
- Examples:
 - `letter-spacing: 2px;`
 - `letter-spacing: 0.1em;`
 - `letter-spacing: normal;`

Word Spacing

word-spacing

- Purpose: Space between words
- Values: `normal` or length
- Examples:
 - `word-spacing: 5px;`
 - `word-spacing: 0.2em;`
 - `word-spacing: normal;`

Line Height

line-height

- Purpose: Height of line boxes
- Values:
 - `normal` - Browser default
 - Number: Multiplier of font size
 - Length: Specific height
 - Percentage: Percentage of font size
- Examples:

- `line-height: 1.5;`
- `line-height: 24px;`
- `line-height: 150%;`

Text Shadow

text-shadow

- Purpose: Adds shadow to text
- Syntax: `text-shadow: h-shadow v-shadow blur-radius color;`
- Examples:
 - `text-shadow: 2px 2px 4px rgba(0,0,0,0.5);`
 - `text-shadow: 1px 1px 2px red;`
 - `text-shadow: 0 0 10px blue;`

Multiple Shadows:

```
text-shadow:
  2px 2px 4px rgba(0,0,0,0.5),
 -1px -1px 2px rgba(255,255,255,0.3);
```

White Space

white-space

- Purpose: Controls whitespace handling
- Values:
 - `normal` - Collapses whitespace (default)
 - `nowrap` - Prevents wrapping
 - `pre` - Preserves whitespace
 - `pre-wrap` - Preserves whitespace but allows wrapping
 - `pre-line` - Preserves line breaks
- Examples:
 - `white-space: nowrap;`
 - `white-space: pre;`

Word Break

word-break

- Purpose: Controls line breaking
- Values:
 - `normal` - Default breaking rules
 - `break-all` - Breaks at any character
 - `keep-all` - Keeps words together
- Example: `word-break: break-all;`

Word Wrap

word-wrap / overflow-wrap

- Purpose: Controls wrapping of long words
- Values:
 - `normal` - Break only at allowed points
 - `break-word` - Break long words if necessary
- Example: `word-wrap: break-word;`

Web Fonts

@font-face Rule

Basic Syntax:

```
@font-face {
  font-family: "CustomFont";
  src: url("font.woff2") format("woff2"),
       url("font.woff") format("woff");
  font-weight: normal;
  font-style: normal;
}
```

Font Formats:

- WOFF2 (Web Open Font Format 2.0)
- WOFF (Web Open Font Format)
- TTF (TrueType Font)
- OTF (OpenType Font)
- EOT (Embedded OpenType)

Google Fonts

Import Method:

```
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;700&display=swap');

body {
  font-family: 'Roboto', sans-serif;
}
```

Link Method (in HTML):

```
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;700&display=swap" rel="stylesheet">
```

8. GLOWING BUTTON V2

Advanced Glowing Button Effect

HTML Structure:

```
<button class="glow-btn">
  <span class="btn-text">Glow Button</span>
</button>
```

CSS Implementation:

```
.glow-btn {
  position: relative;
  padding: 15px 30px;
  background: linear-gradient(45deg, #ff006e, #8338ec);
  border: none;
  border-radius: 50px;
  color: white;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
  overflow: hidden;
  transition: all 0.3s ease;
  box-shadow:
    0 0 20px rgba(255, 0, 110, 0.4),
    0 0 40px rgba(131, 56, 236, 0.3),
    inset 0 0 20px rgba(255, 255, 255, 0.1);
}

.glow-btn::before {
  content: '';
  position: absolute;
  top: -50%;
  left: -50%;
  width: 200%;
  height: 200%;
  background: linear-gradient(45deg, transparent, rgba(255, 255, 255, 0.3), transparent);
  transform: rotate(45deg);
  transition: all 0.5s;
  opacity: 0;
}
```

```
.glow-btn:hover {
  transform: translateY(-2px);
  box-shadow:
    0 0 30px rgba(255, 0, 110, 0.6),
    0 0 60px rgba(131, 56, 236, 0.5),
    0 10px 30px rgba(0, 0, 0, 0.2),
    inset 0 0 30px rgba(255, 255, 255, 0.2);
}

.glow-btn:hover::before {
  opacity: 1;
  animation: shine 0.5s ease-in-out;
}

.glow-btn:active {
  transform: translateY(0);
  box-shadow:
    0 0 20px rgba(255, 0, 110, 0.8),
    0 0 40px rgba(131, 56, 236, 0.7),
    inset 0 0 20px rgba(255, 255, 255, 0.3);
}

.btn-text {
  position: relative;
  z-index: 1;
}

@keyframes shine {
  0% {
    transform: translateX(-100%) rotate(45deg);
  }
  100% {
    transform: translateX(100%) rotate(45deg);
  }
}
```

Pulsing Glow Effect

```
.pulse-glow {
  animation: pulse-glow 2s ease-in-out infinite alternate;
}

@keyframes pulse-glow {
  from {
    box-shadow:
      0 0 20px rgba(255, 0, 110, 0.4),
      0 0 40px rgba(131, 56, 236, 0.3);
  }
  to {
    box-shadow:
      0 0 40px rgba(255, 0, 110, 0.8),
      0 0 80px rgba(131, 56, 236, 0.6);
  }
}
```

Neon Text Effect

```

.neon-text {
  color: #fff;
  text-shadow:
    0 0 5px #ff006e,
    0 0 10px #ff006e,
    0 0 15px #ff006e,
    0 0 20px #ff006e,
    0 0 35px #ff006e,
    0 0 40px #ff006e;
  animation: neon-flicker 1.5s ease-in-out infinite alternate;
}

@keyframes neon-flicker {
  0%, 18%, 22%, 25%, 53%, 57%, 100% {
    text-shadow:
      0 0 5px #ff006e,
      0 0 10px #ff006e,
      0 0 15px #ff006e,
      0 0 20px #ff006e,
      0 0 35px #ff006e,
      0 0 40px #ff006e;
  }
  20%, 24%, 55% {
    text-shadow: none;
  }
}

```

9. GLOWING BUTTONS EFFECT

Basic Glowing Button

HTML:

```
<button class="glow-button">Click Me</button>
```

CSS:

```

.glow-button {
  padding: 12px 24px;
  background: #0066cc;
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 16px;
  cursor: pointer;
  transition: all 0.3s ease;
  box-shadow: 0 0 10px rgba(0, 102, 204, 0.5);
}

.glow-button:hover {
  box-shadow: 0 0 20px rgba(0, 102, 204, 0.8);
  transform: translateY(-2px);
}

```

RGB Glowing Button

```
.rgb-glow {
  background: linear-gradient(45deg, #ff0000, #00ff00, #0000ff);
  background-size: 300% 300%;
  animation: rgb-shift 3s ease-in-out infinite;
  box-shadow: 0 0 20px rgba(255, 255, 255, 0.5);
}

@keyframes rgb-shift {
  0% {
    background-position: 0% 50%;
  }
  50% {
    background-position: 100% 50%;
  }
  100% {
    background-position: 0% 50%;
  }
}
```

Glowing Border Button

```
.glow-border {
  background: transparent;
  border: 2px solid #00ff88;
  color: #00ff88;
  position: relative;
  overflow: hidden;
}

.glow-border::before {
  content: '';
  position: absolute;
  top: 0;
  left: -100%;
  width: 100%;
  height: 100%;
  background: linear-gradient(90deg, transparent, rgba(0, 255, 136, 0.4), transparent);
  transition: left 0.5s;
}

.glow-border:hover::before {
  left: 100%;
}

.glow-border:hover {
  box-shadow:
    0 0 20px rgba(0, 255, 136, 0.6),
    inset 0 0 20px rgba(0, 255, 136, 0.1);
}
```

Multi-Color Glow

```
.multi-glow {
  background: #1a1a1a;
  border: 1px solid #333;
  color: white;
  box-shadow:
    0 0 10px rgba(255, 0, 150, 0.5),
    0 0 20px rgba(0, 255, 255, 0.3),
    0 0 30px rgba(255, 255, 0, 0.2);
  animation: multi-glow 3s ease-in-out infinite;
}

@keyframes multi-glow {
  0%, 100% {
    box-shadow:
      0 0 10px rgba(255, 0, 150, 0.5),
      0 0 20px rgba(0, 255, 255, 0.3),
      0 0 30px rgba(255, 255, 0, 0.2);
  }
  33% {
    box-shadow:
      0 0 10px rgba(0, 255, 255, 0.5),
      0 0 20px rgba(255, 255, 0, 0.3),
      0 0 30px rgba(255, 0, 150, 0.2);
  }
  66% {
    box-shadow:
      0 0 10px rgba(255, 255, 0, 0.5),
      0 0 20px rgba(255, 0, 150, 0.3),
      0 0 30px rgba(0, 255, 255, 0.2);
  }
}
```

10. GLOWING NAVBAR V1

Basic Glowing Navigation

HTML Structure:

```
<nav class="glow-navbar">
  <div class="nav-container">
    <div class="nav-logo">
      <a href="#" class="logo-link">Logo</a>
    </div>
    <ul class="nav-menu">
      <li class="nav-item">
        <a href="#" class="nav-link active">Home</a>
      </li>
      <li class="nav-item">
        <a href="#" class="nav-link">About</a>
      </li>
      <li class="nav-item">
        <a href="#" class="nav-link">Services</a>
      </li>
      <li class="nav-item">
        <a href="#" class="nav-link">Contact</a>
      </li>
    </ul>
  </div>
</nav>
```

CSS Implementation:

```
.glow-navbar {
  background: rgba(0, 0, 0, 0.9);
  backdrop-filter: blur(10px);
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  box-shadow: 0 2px 20px rgba(0, 255, 255, 0.2);
  position: sticky;
  top: 0;
  z-index: 1000;
}

.nav-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 70px;
}

.nav-logo .logo-link {
  color: #00ff88;
  font-size: 24px;
  font-weight: bold;
  text-decoration: none;
  text-shadow: 0 0 10px rgba(0, 255, 136, 0.5);
  transition: all 0.3s ease;
}

.nav-logo .logo-link:hover {
  text-shadow: 0 0 20px rgba(0, 255, 136, 0.8);
}

.nav-menu {
  display: flex;
  list-style: none;
  margin: 0;
  padding: 0;
  gap: 30px;
}

.nav-link {
  color: #ffffff;
  text-decoration: none;
  font-size: 16px;
  font-weight: 500;
  padding: 10px 20px;
  border-radius: 25px;
  transition: all 0.3s ease;
  position: relative;
  overflow: hidden;
}

.nav-link::before {
  content: '';
  position: absolute;
  top: 0;
  left: -100%;
  width: 100%;
  height: 100%;
  background: linear-gradient(90deg, transparent, rgba(0, 255, 255, 0.2), transparent);
  transition: left 0.5s;
}

.nav-link:hover::before {
  left: 0;
}
```

```
    left: 100%;
  }

.nav-link:hover {
  color: #00ffff;
  box-shadow:
    0 0 10px rgba(0, 255, 255, 0.5),
    inset 0 0 10px rgba(0, 255, 255, 0.1);
  transform: translateY(-2px);
}

.nav-link.active {
  color: #00ff88;
  box-shadow:
    0 0 15px rgba(0, 255, 136, 0.6),
    inset 0 0 15px rgba(0, 255, 136, 0.1);
  background: rgba(0, 255, 136, 0.1);
}
```

Animated Underline Effect

```
.nav-link-underline {
  position: relative;
}

.nav-link-underline::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 0;
  width: 0;
  height: 2px;
  background: linear-gradient(90deg, #00ff88, #00ffff);
  transition: width 0.3s ease;
  box-shadow: 0 0 10px rgba(0, 255, 136, 0.5);
}

.nav-link-underline:hover::after {
  width: 100%;
}
```

Responsive Glowing Navbar

```
.nav-toggle {
  display: none;
  background: none;
  border: none;
  color: #00ff88;
  font-size: 20px;
  cursor: pointer;
}

@media (max-width: 768px) {
  .nav-toggle {
    display: block;
  }

  .nav-menu {
    position: fixed;
    top: 70px;
    left: -100%;
    width: 100%;
    height: calc(100vh - 70px);
    background: rgba(0, 0, 0, 0.95);
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    padding-top: 50px;
    transition: left 0.3s ease;
  }

  .nav-menu.active {
    left: 0;
  }

  .nav-item {
    margin: 20px 0;
  }
}
```

11. GRID

Grid Container Setup

display: grid

- Purpose: Creates a grid container
- Values: grid or inline-grid
- Example: display: grid;

Grid Template Columns

grid-template-columns

- Purpose: Defines columns in grid
- Values:
 - Length: 200px 100px 300px
 - Percentages: 25% 50% 25%
 - fr units: 1fr 2fr 1fr
 - repeat(): repeat(3, 1fr)
 - auto: auto
 - min-content/max-content: min-content max-content
- Examples:
 - grid-template-columns: 1fr 1fr 1fr;
 - grid-template-columns: repeat(3, 1fr);
 - grid-template-columns: 200px 1fr 100px;

Grid Template Rows

grid-template-rows

- Purpose: Defines rows in grid
- Values: Same as grid-template-columns
- Examples:
 - `grid-template-rows: 100px 200px;`
 - `grid-template-rows: repeat(3, 1fr);`
 - `grid-template-rows: auto 1fr auto;`

Grid Template Areas

grid-template-areas

- Purpose: Defines named grid areas
- Values: String of area names
- Example:

```
grid-template-areas:  
"header header header"  
"sidebar main main"  
"footer footer footer";
```

Grid Template (Shorthand)

grid-template

- Purpose: Shorthand for columns, rows, and areas
- Syntax: `grid-template: rows / columns;`
- Example: `grid-template: 100px 1fr 50px / 200px 1fr;`

Grid Gap

gap

- Purpose: Space between grid items
- Values: Length values
- Examples:
 - `gap: 20px;`
 - `gap: 10px 20px;` (row-gap column-gap)

row-gap

- Purpose: Space between rows
- Example: `row-gap: 15px;`

column-gap

- Purpose: Space between columns
- Example: `column-gap: 20px;`

Grid Auto Columns

grid-auto-columns

- Purpose: Size of auto-generated columns
- Values: Length, percentage, fr, auto
- Example: `grid-auto-columns: 100px;`

Grid Auto Rows

grid-auto-rows

- Purpose: Size of auto-generated rows
- Values: Length, percentage, fr, auto
- Example: `grid-auto-rows: 150px;`

Grid Auto Flow

grid-auto-flow

- Purpose: Controls auto-placement of items
- Values:

- `row` - Fill rows first (default)
- `column` - Fill columns first
- `dense` - Fill gaps
- `row dense` - Fill rows densely
- `column dense` - Fill columns densely
- Example: `grid-auto-flow: column;`

Grid Item Properties

Grid Column Start/End

`grid-column-start`

- Purpose: Starting column line
- Values: Line number, span, auto
- Example: `grid-column-start: 2;`

`grid-column-end`

- Purpose: Ending column line
- Values: Line number, span, auto
- Example: `grid-column-end: 4;`

`grid-column` (Shorthand)

- Purpose: Start and end column lines
- Syntax: `grid-column: start / end;`
- Examples:
 - `grid-column: 1 / 3;`
 - `grid-column: 2 / span 2;`

Grid Row Start/End

`grid-row-start`

- Purpose: Starting row line
- Values: Line number, span, auto
- Example: `grid-row-start: 1;`

`grid-row-end`

- Purpose: Ending row line
- Values: Line number, span, auto
- Example: `grid-row-end: 3;`

`grid-row` (Shorthand)

- Purpose: Start and end row lines
- Syntax: `grid-row: start / end;`
- Examples:
 - `grid-row: 1 / 3;`
 - `grid-row: 2 / span 1;`

Grid Area

`grid-area`

- Purpose: Assigns item to named area or specifies position
- Values: Area name or row-start/column-start/row-end/column-end
- Examples:
 - `grid-area: header;`
 - `grid-area: 1 / 2 / 3 / 4;`

Grid Alignment

Justify Items

`justify-items`

- Purpose: Horizontal alignment of items within cells
- Values:
 - `start` - Align to start
 - `end` - Align to end
 - `center` - Center items
 - `stretch` - Stretch to fill (default)
- Example: `justify-items: center;`

Align Items

align-items

- Purpose: Vertical alignment of items within cells
- Values: Same as justify-items
- Example: `align-items: center;`

Place Items

place-items

- Purpose: Shorthand for align-items and justify-items
- Syntax: `place-items: align-items justify-items;`
- Example: `place-items: center;`

Justify Content

justify-content

- Purpose: Horizontal alignment of grid within container
- Values:
 - `start` - Align to start
 - `end` - Align to end
 - `center` - Center grid
 - `stretch` - Stretch grid
 - `space-around` - Space around tracks
 - `space-between` - Space between tracks
 - `space-evenly` - Even space distribution
- Example: `justify-content: center;`

Align Content

align-content

- Purpose: Vertical alignment of grid within container
- Values: Same as justify-content
- Example: `align-content: center;`

Place Content

place-content

- Purpose: Shorthand for align-content and justify-content
- Syntax: `place-content: align-content justify-content;`
- Example: `place-content: center;`

13. Margins

Overview

The `margin` property in CSS is used to create space around elements, outside of any defined borders. It controls the spacing between an element and surrounding elements.

📄 Properties

Property	Description
<code>margin</code>	Shorthand for setting all four margins (top, right, bottom, left)
<code>margin-top</code>	Sets top margin
<code>margin-right</code>	Sets right margin
<code>margin-bottom</code>	Sets bottom margin
<code>margin-left</code>	Sets left margin

Syntax

```
/* One value applies to all sides */
margin: 10px;

/* Two values: vertical | horizontal */
margin: 10px 20px;

/* Three values: top | horizontal (left/right) | bottom */
margin: 10px 20px 30px;

/* Four values: top | right | bottom | left (clockwise) */
margin: 10px 20px 30px 40px;
```

Special Values

- `auto`: Automatically calculated by browser (used for centering)
- `%`: Percentage relative to parent width
- Negative values: Pulls element outward

🔧 Tips

- Use `margin: 0 auto;` to **center block-level elements horizontally**.
- Collapsing margins: Vertical margins between adjacent elements may collapse into one.

14. Positioning

Overview

CSS positioning allows you to control how an element is placed in the document layout. The `position` property determines how an element is positioned on the page.

🔧 Properties

Property	Description
<code>position</code>	Specifies the type of positioning method used (<code>static</code> , <code>relative</code> , <code>absolute</code> , <code>fixed</code> , <code>sticky</code>)
<code>top</code> , <code>right</code> , <code>bottom</code> , <code>left</code>	Define position when <code>position</code> is not <code>static</code>
<code>z-index</code>	Controls stacking order of positioned elements
<code>clip</code>	Clips absolutely positioned elements (rarely used now)
<code>transform</code>	Can be used with positioning for advanced layout

Syntax

```
position: static | relative | absolute | fixed | sticky;
```

Types of Positioning

1. `static` (Default)

- Default positioning.
- Cannot use `top`, `bottom`, `left`, or `right`.

```
position: static;
```

2. `relative`

- Positioned relative to its normal position.
- Does **not take it out of the document flow**.


```
position: relative;
top: 10px;
left: 20px;
```

3. absolute

- Positioned relative to the nearest positioned ancestor (`relative` , `absolute` , `fixed` , or `sticky`).
- If none, goes to viewport origin.
- **Takes the element out of the document flow.**

```
position: absolute;
top: 50px;
left: 100px;
```

4. fixed

- Positioned relative to the viewport.
- Remains in place even when scrolling.

```
position: fixed;
top: 0;
right: 0;
```

5. sticky

- Hybrid of `relative` and `fixed` .
- Sticks to a given position during scroll after reaching threshold.

```
position: sticky;
top: 10px;
```

⚠ Must specify at least `top` or `bottom` for `sticky` to work.

z-index

- Only works on positioned elements (`relative` , `absolute` , `fixed` , `sticky`).
- Higher number = closer to user.

```
z-index: 999;
```

15. Pseudo Classes

Overview

Pseudo-classes are keywords added to selectors that specify a special state of the selected element. They start with a colon `:` .

Common Pseudo Classes

Pseudo Class	Description
<code>:link</code>	Unvisited link
<code>:visited</code>	Visited link
<code>:hover</code>	Mouse hover
<code>:active</code>	Active element (while clicked)
<code>:focus</code>	Element has focus (e.g., input field)
<code>:first-child</code>	First child of its parent

:last-child Pseudo Class	Last child of its parent Description
<code>:nth-child(n)</code>	nth child of its parent
<code>:nth-of-type(n)</code>	nth child of specific type
<code>:only-child</code>	Only child of its parent
<code>:empty</code>	Elements with no children
<code>:checked</code>	Checked form elements (radio, checkbox)
<code>:enabled</code> / <code>:disabled</code>	Form elements enabled/disabled
<code>:required</code> / <code>:optional</code>	Required/optional form fields
<code>:target</code>	Targeted element via URL fragment
<code>:lang(language)</code>	Element with matching language attribute
<code>:not(selector)</code>	Negates a selector
<code>:root</code>	Represents the root element (HTML tag)
<code>:first-of-type</code>	First of its type among siblings
<code>:last-of-type</code>	Last of its type among siblings
<code>:only-of-type</code>	Only one of its type among siblings

Syntax

```
selector:pseudo-class {
  property: value;
}
```

Examples

Basic Interaction

```
a:hover {
  color: red;
}

input:focus {
  border-color: blue;
}
```

Structural Selectors

```
li:first-child {
  font-weight: bold;
}

tr:nth-child(even) {
  background-color: #f2f2f2;
}
```

Form States

```
input:required {
  border: 2px solid orange;
}

input:checked + label {
  color: green;
}
```

Advanced Usage

```
div:not(.special) {
  color: gray;
}

p:lang(en) {
  quotes: "\201C" "\201D";
}
```

Bonus: Pseudo Elements vs Pseudo Classes

Feature	Pseudo Class	Pseudo Element
Syntax	Single colon <code>:hover</code>	Double colon <code>::before</code>
Purpose	Defines a special state	Adds virtual elements to content
Example	<code>a:hover</code> , <code>li:nth-child(2)</code>	<code>::before</code> , <code>::after</code> , <code>::marker</code>

Pseudo Elements

- `::before`
- `::after`
- `::first-line`
- `::first-letter`
- `::selection`
- `::placeholder`
- `::marker` (for list items)
- `::file-selector-button` (for `<input type="file">` buttons)

🔗 Example Combos

Centering with Margin & Positioning

```
.center-block {
  width: 200px;
  margin: 0 auto;
}

.absolute-center {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

Hover Effect on Button

```
button {
  background: #007bff;
  transition: background 0.3s;
}

button:hover {
  background: #0056b3;
}
```

Style Odd Rows in Table

```
tr:nth-child(odd) {
  background-color: #fff3cd;
}
```

Summary

Topic	Key Takeaways
Margins	Control spacing around elements. Understand collapsing margins and <code>auto</code> for centering.
Positioning	Choose between <code>static</code> , <code>relative</code> , <code>absolute</code> , <code>fixed</code> , <code>sticky</code> . Know how they affect layout and each other.
Pseudo Classes	Use to style dynamic states and select elements based on structure or interaction. Essential for interactive UI.